Name: **Asad Ali**
Instructor: **Sharjeel Khan**

# CyberSecurity-Task 01

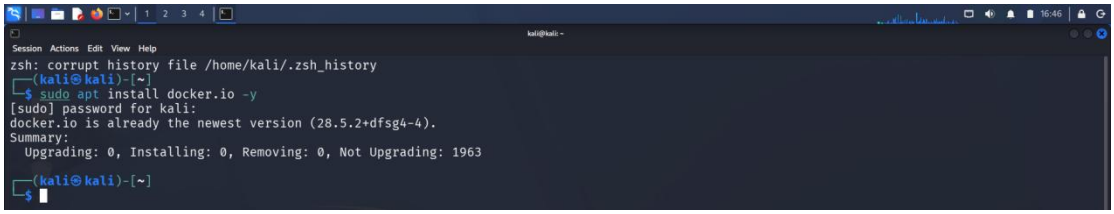
# A) Environment Setup

OWASP Juice Shop can be deployed on a Kali Linux machine using two approaches: pulling and running its Docker image or cloning the project directly from its official GitHub repository. For this assessment, Docker was selected as the preferred deployment method because it simplifies installation, minimizes manual configuration, and provides a consistent environment without requiring the application and its dependencies to be configured separately. This approach allows the vulnerable web application to be deployed efficiently and accessed locally for authorized security testing.

A) Installing the docker

To deploy the owasp juice shop via docker the very first step is to install docker in your machine. To install docker following command is used shown in Picture below.

**POC**

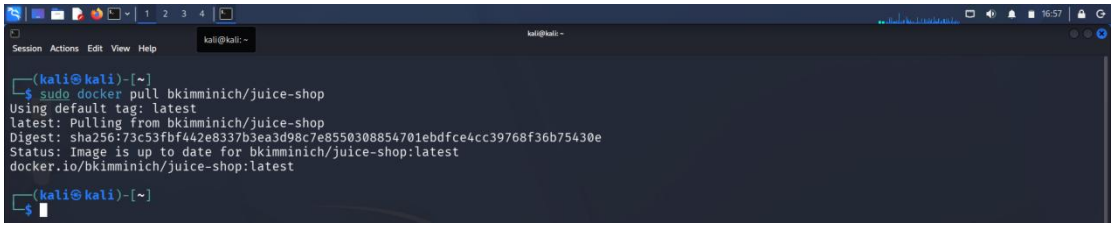



Confirm the docker installation by checking its version



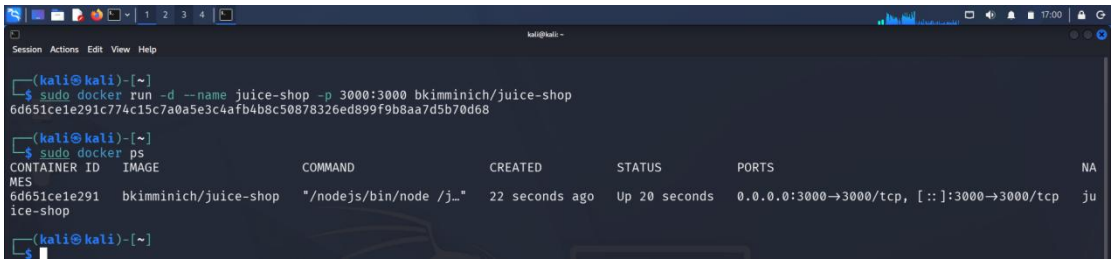

B) Deploying owasp juice shop via docker image

Pulling the owasp juice shop Image shown in Figure below.



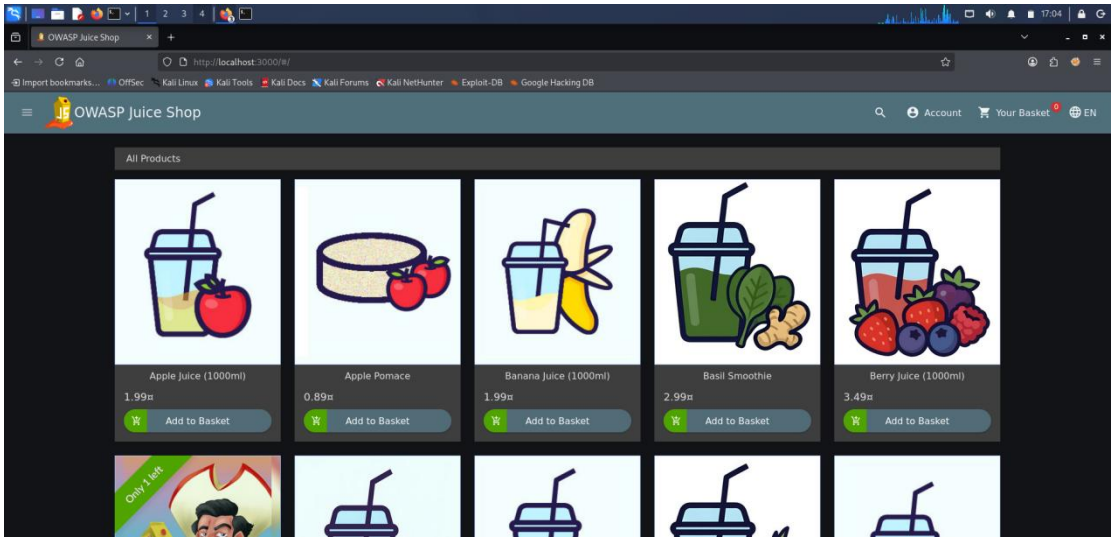
C) Starting the application

To start the application we need to run the contianer from Image shown in figure below.




D) Verifying that application is accessible via url

Now to verify that application is accessible we need to visit the url http://localhost:3000.



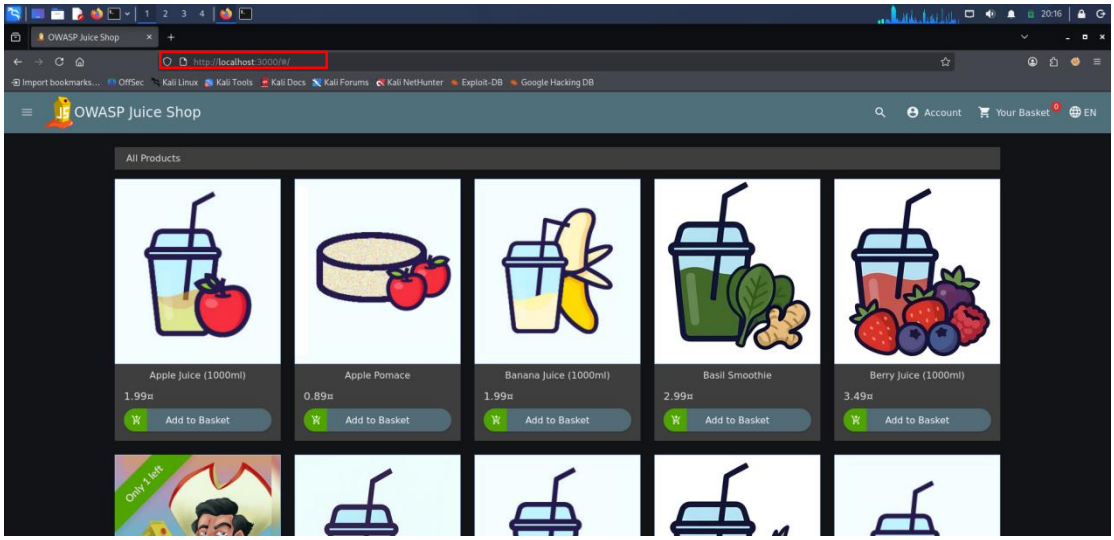

In the above Figure it is clearly shown that aplication is accessible via url that means application is deployed successfully via docker.

# B) Attack Surface Mapping

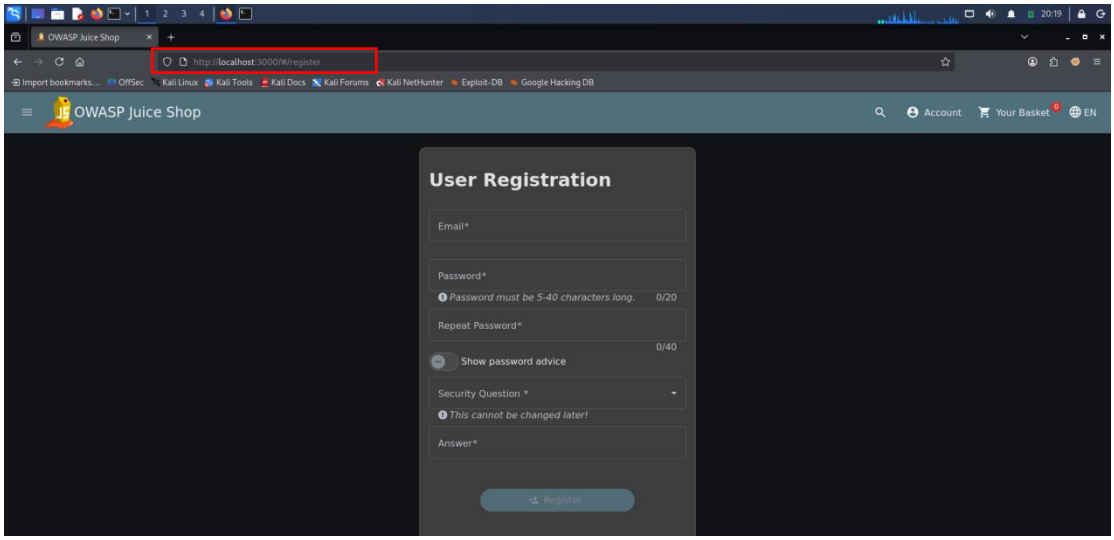
## 1 Web Application Discovery

### a) Publicly Accessible Pages

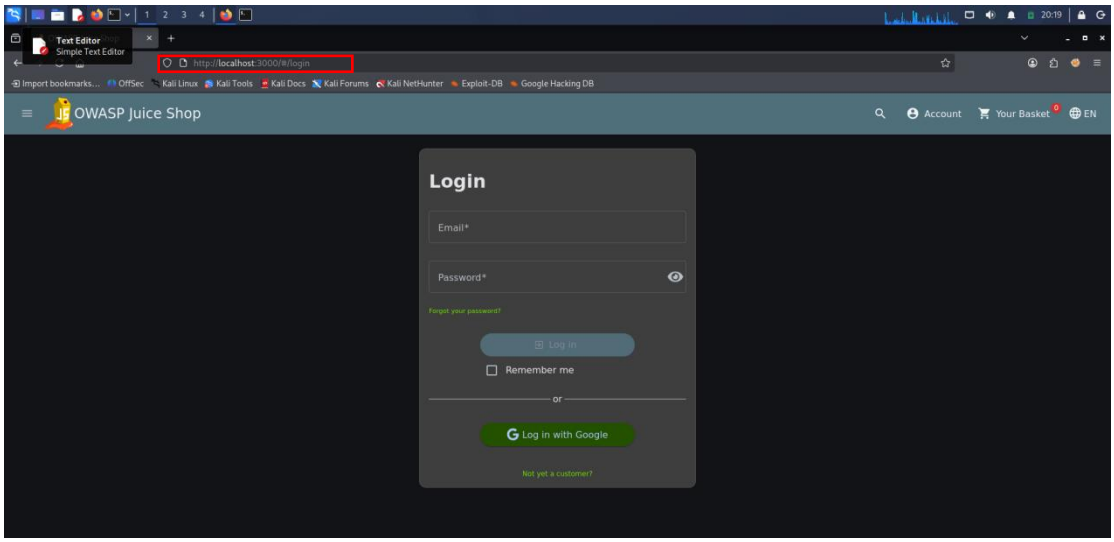
**Main Page  is Accessible without Login**



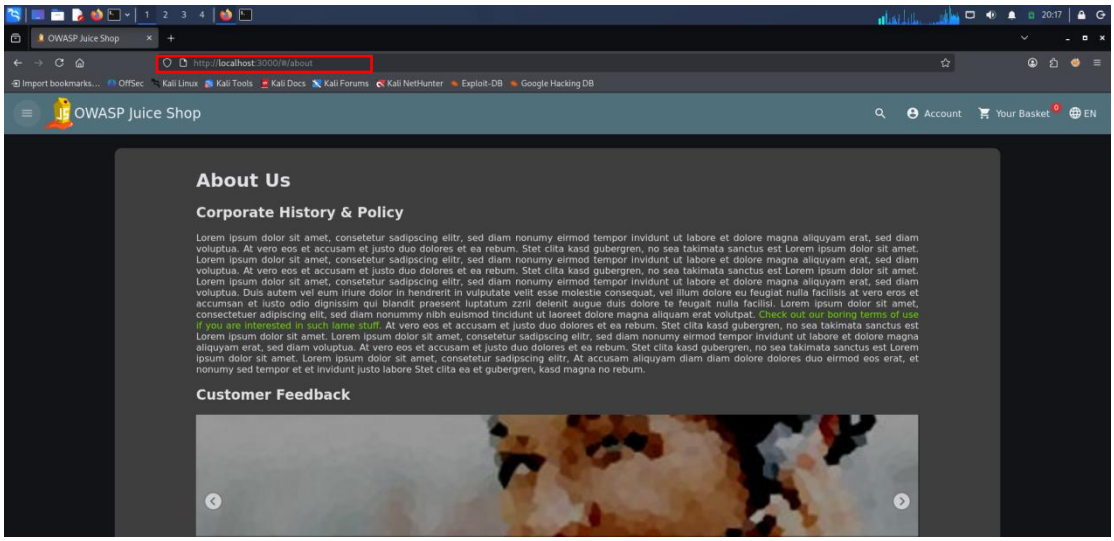

**Register Page is Accessible without Login**



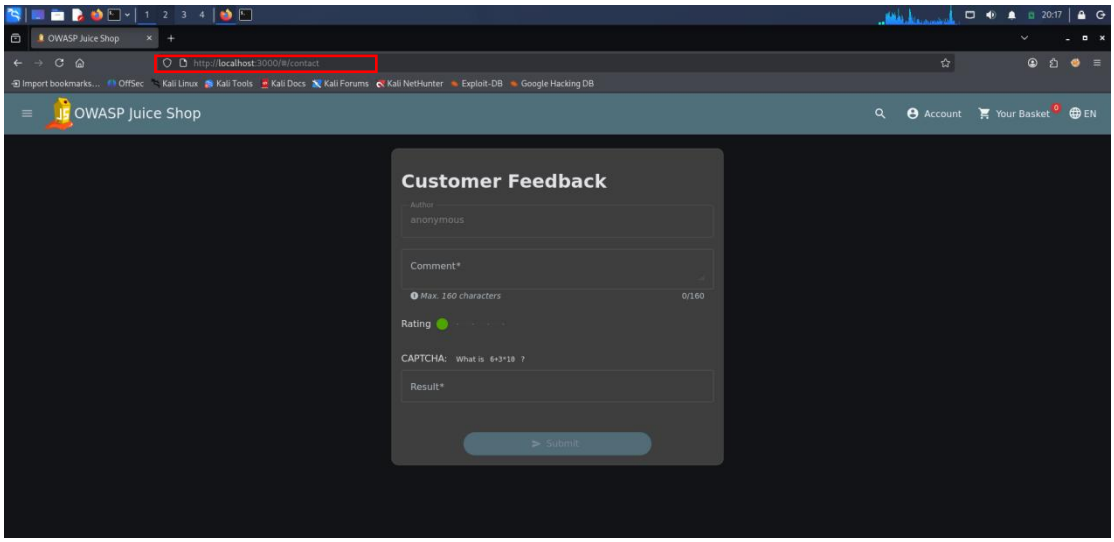
## Login Page is Accessible without Login



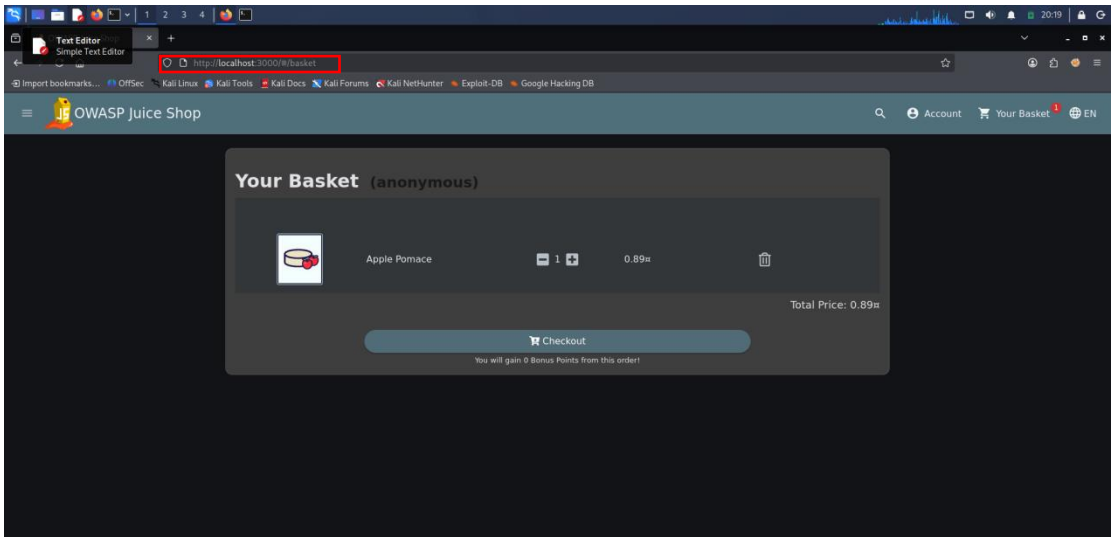

## About Page is Accessible without Login



## Contact Page is Accessible without Login




## Basket Page is Accessible without Login




| S.No | Route | Accessibility |
|---|---|---|
| 1 | / | Public |
| 2 | /register | Public |
| 3 | /login | Public |
| 4 | /about | Public |
| 5 | /contact | Public |
| 6 | /basket | Public |

All Publicly Accessible web Pages are listed above in the table

## b) Application routes

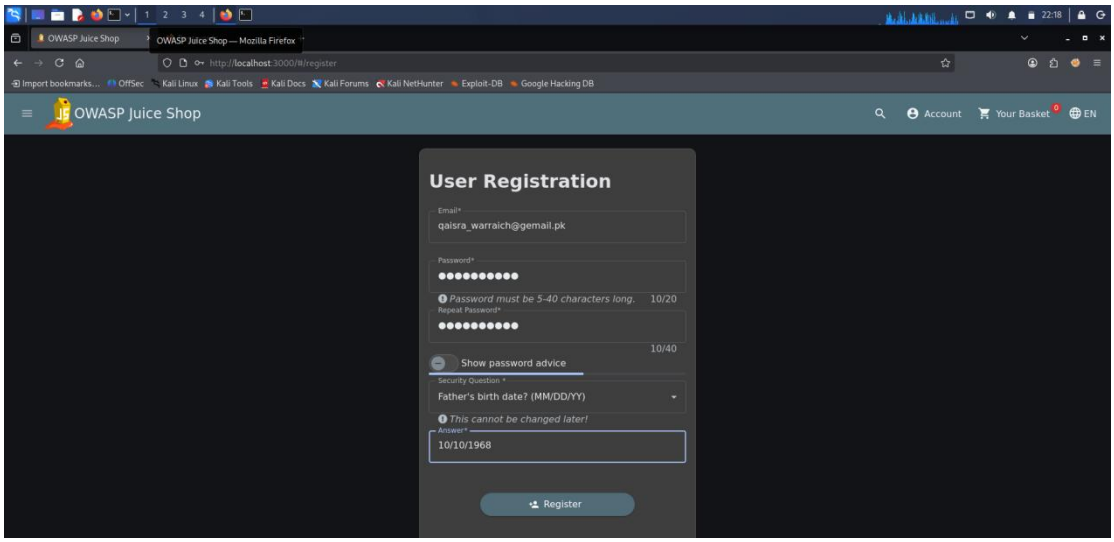
| S.No | Route | Authentication |
|---|---|---|
| 1 | / | yes/no |
| 2 | /profile | yes |
| 3 | /search | yes/no |
| 4 | /recycle | yes |
| 5 | /saved-payment-methods | yes |
| 6 | /privacy-security/two-factor-authentication | yes |
| 7 | wallet | yes |
| 8 | wallet-web3 | yes |
| 9 | /privacy-security/privacy-policy | yes |
| 10 | /order-history | yes |
| 11 | /privacy-security/last-login-ip | yes |
| 12 | /privacy-security/data-export | yes |
| 13 | /dataerasure | yes |
| 14 | /privacy-security/change-password | yes |
| 15 | /address/select | yes |
| 16 | /address/saved | yes |

## c) Authenticated-related Pages

The following pages were identified as being directly related to user authentication and account security. These include account registration and login, profile management, password management, two-factor authentication, and login activity information.

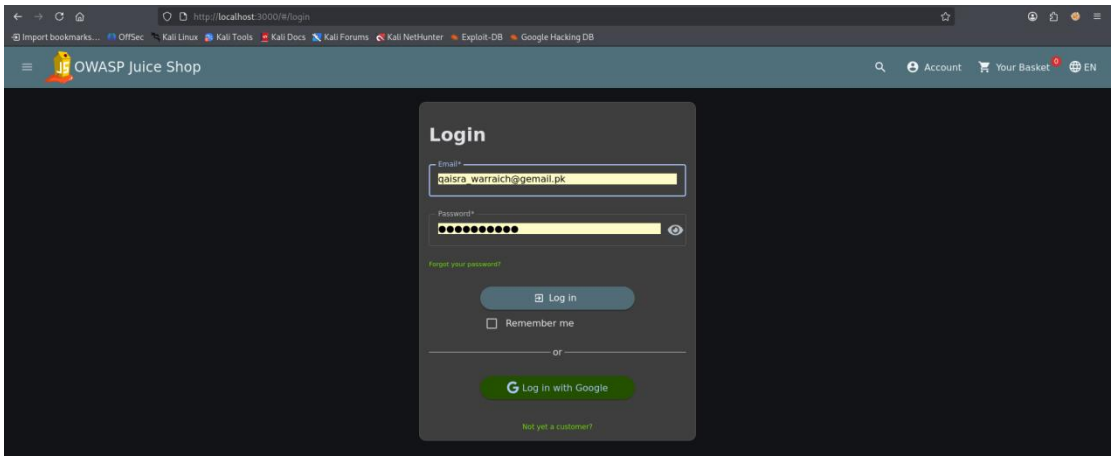
| S.No | Route | Purpose |
|---|---|---|
| 1 | /register | Account registeration |
| 2 | /login | User authentication |
| 3 | /profile | Profile management |
| 4 | /privacy-security/change-password | Password management |
| 5 | /privacy-security/two-factor-authentication | Two factor authentication |

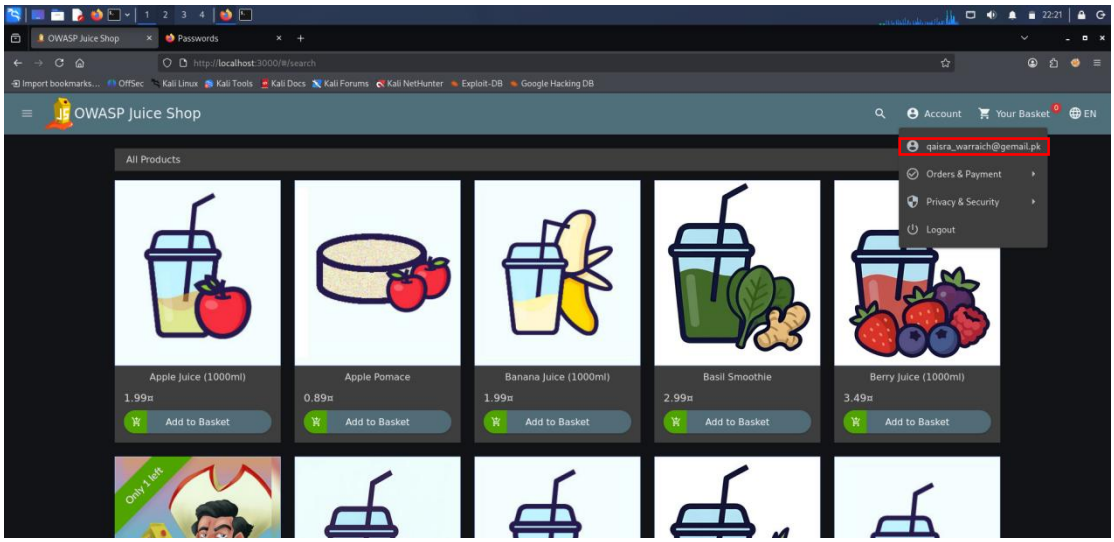
## POC of Register Page




In the above figure it is shown that a test account is crated with email and password.

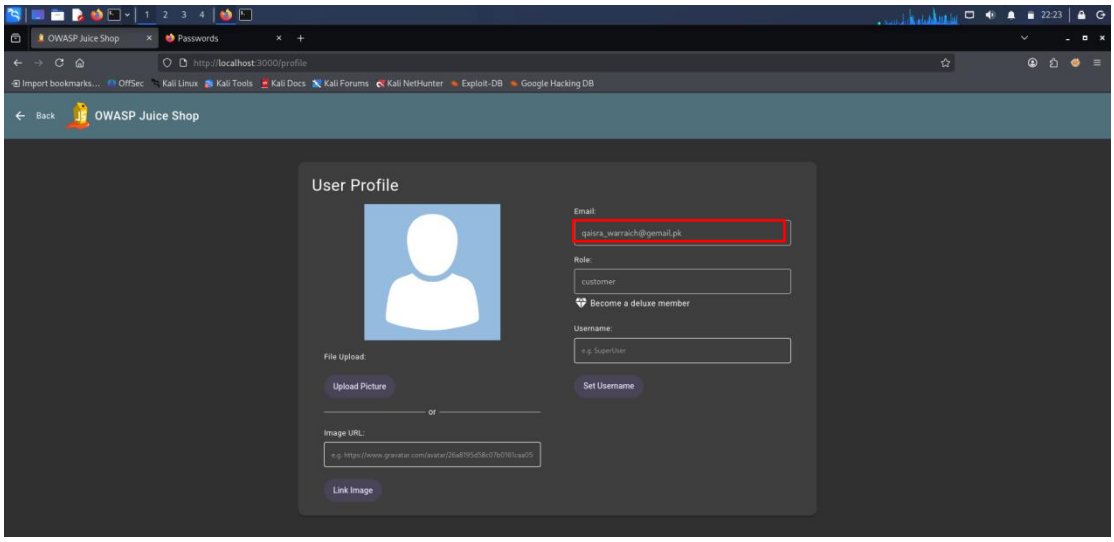
## POC of login Page



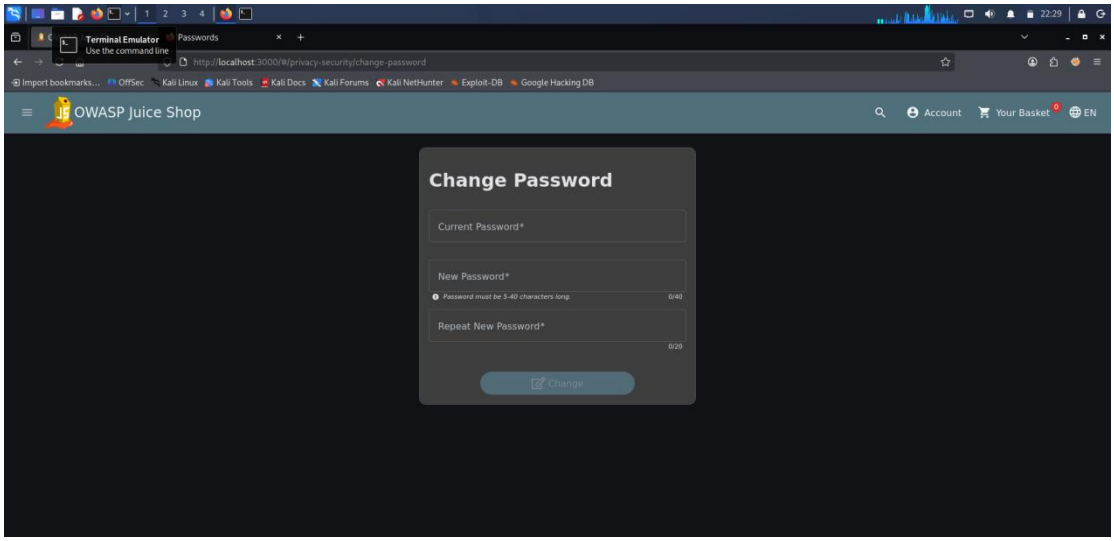
In the above figure it is shown that a test account is login with email and password.

**POC of Profile Page**




In the above figure it is shown that profile page is accessed with the account email we loggedIn.

**POC of Password management**



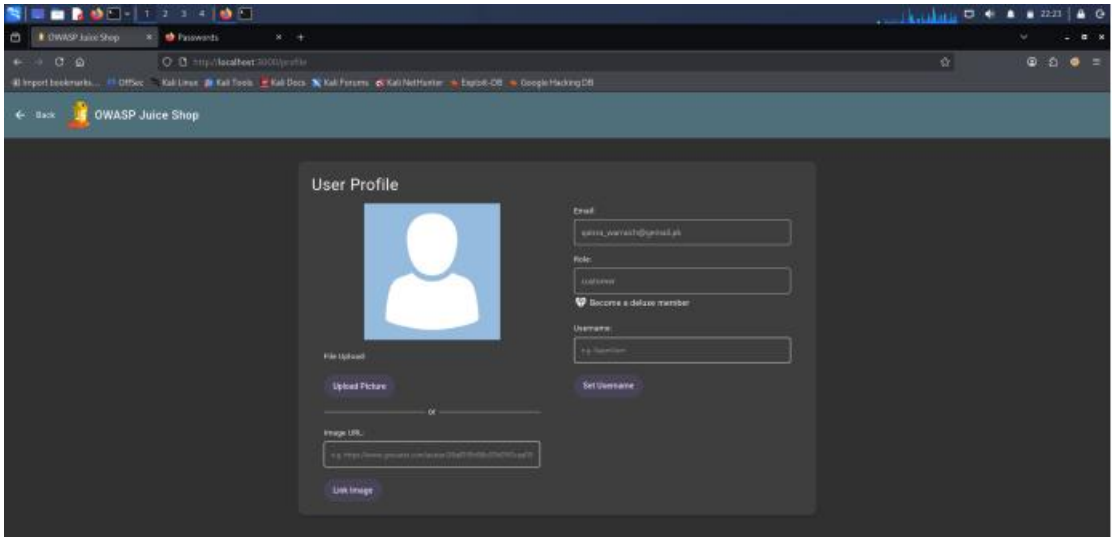

In the above figure it is shown that change password page is accessed.

## d) User functionality

The authenticated application was explored using an authorized test account to identify the functionality available to a normal user. The following table summarizes the main user-facing functions identified during the assessment and their corresponding application routes.

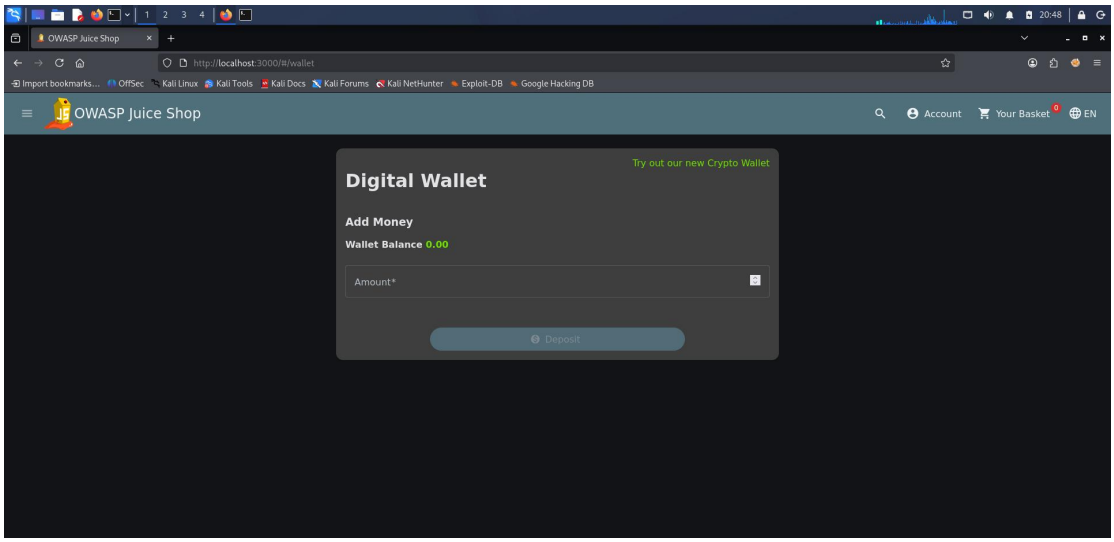
| S.No | User-Functionality | Related-route |
|---|---|---|
| 1 | Search | /search |
| 2 | Manage profile | /profile |
| 3 | Manage wallet | /wallet |
| 4 | Manage web3 wallet | /wallet-web3 |
| 5 | Manage saved payment methods | /saved-payment-methods |
| 6 | View Order history | /order-history |
| 7 | Managed saved addresses | /address/saved |
| 8 | Select address | /address/select |
| 9 | Configring 2FA | /privacy-security/two-factor-authentication |
| 10 | Change password | /privacy-security/change-password |
| 11 | View last-login IP | /privacy-security/last-login-ip |
| 12 | Export personal data | /privacy-security/data-export |
| 13 | Request data erasure | dataerasure |
| 14 | Access Privacy information | /privacy-security/privacy-policy |
| 15 | Recycle functionality | /recycle |

**POC of Profile Page**



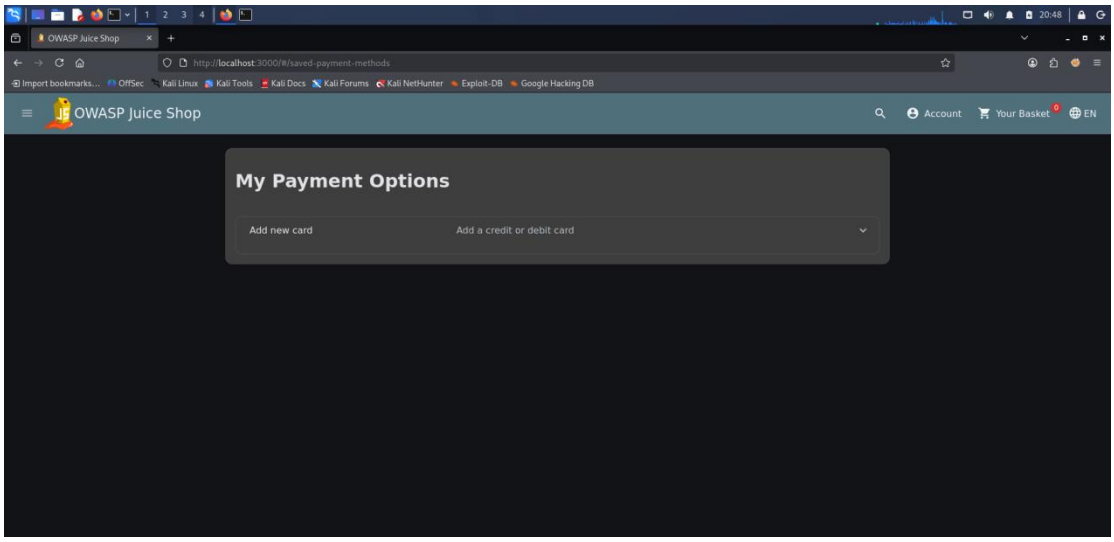

In the above figure it is shown that profile page is accessed with the account email we loggedIn.

## POC of Wallet



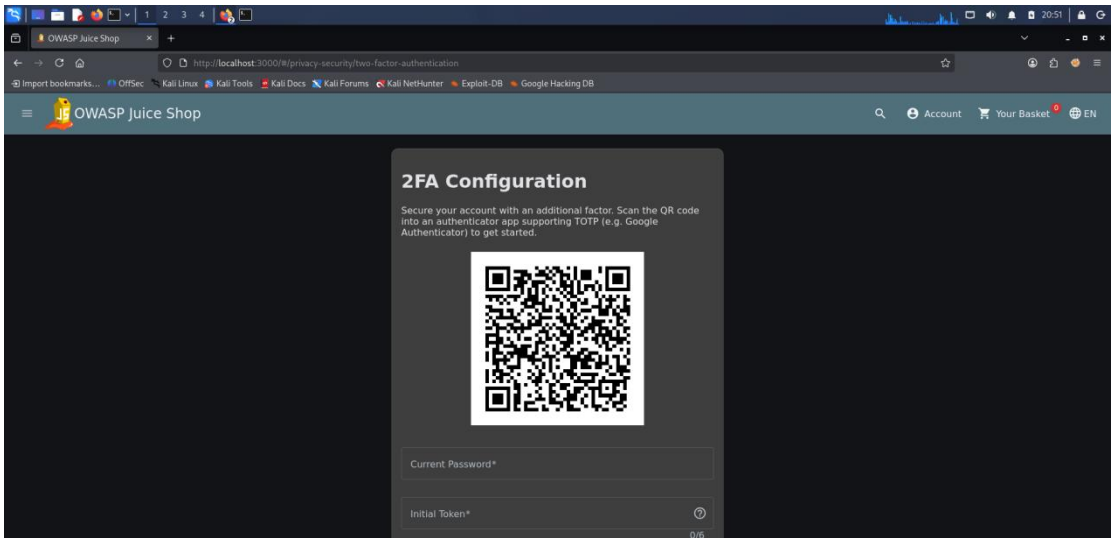

In the above figure it is shown that wallet page is accessed.

## POC of saved-payment-methods



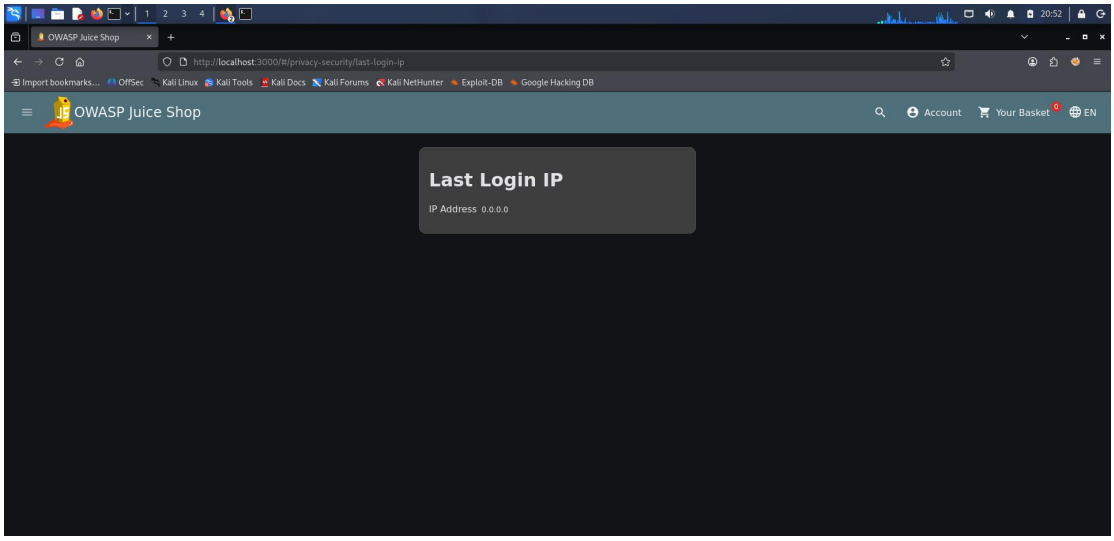

In the above figure it is shown that saved-payment-methods page is accessed.

**POC of Configuring 2FA**




In the above figure it is shown that two-factor-authentication  page is accessed.

**POC of View last-login IP**




In the above figure it is shown that last-login-ip page is accessed.

## e) Administrative functionality

Administrative functionality was not assessed because administrator credentials were not provided. No authentication bypass or privilege escalation was attempted, as exploitation was outside the scope of this discovery activity.

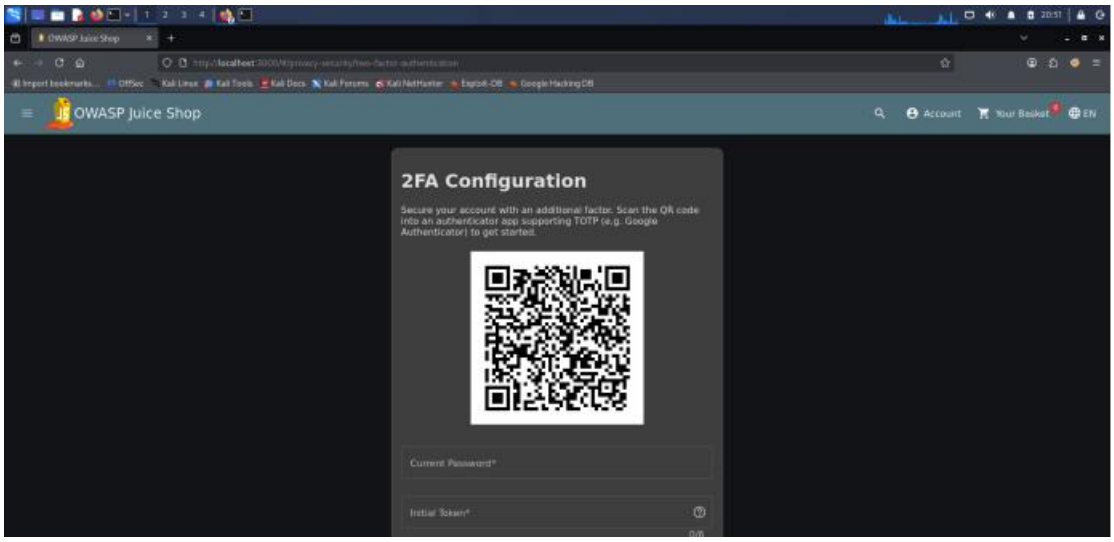
## f) Interesting resources

The  table below identifies resources considered particularly relevant for security assessment based on the type of functionality and information they handle. These resources include financial, authentication, account-management, and personal-data

functions, which may represent higher-value areas of the application's attack surface. Identifying these resources helps prioritize subsequent authorized security testing and determine where security controls should be examined more closely.

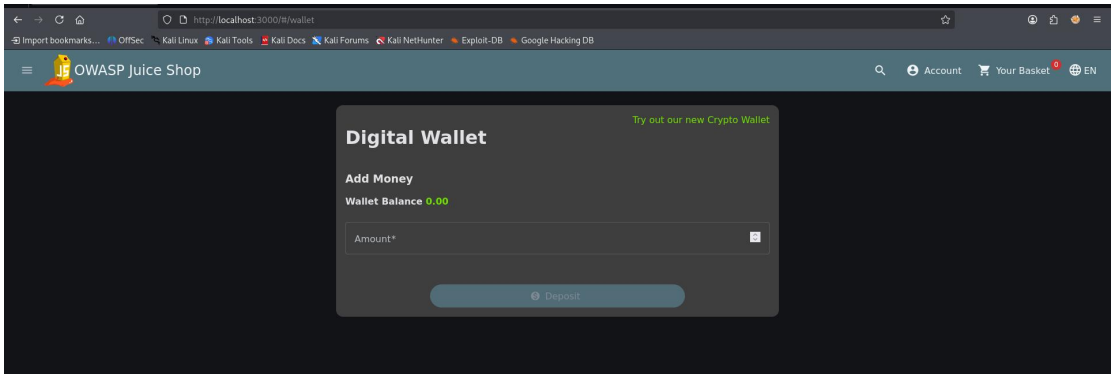
| S.No | route |
|---|---|
| 1 | /wallet |
| 2 | /wallet-web3 |
| 3 | /saved-payment-methods |
| 4 | /order-history |
| 5 | /address/saved |
| 6 | /privacy-security/two-factor-authentication |
| 7 | /privacy-security/change-password |
| 8 | /privacy-security/last-login-ip |
| 9 | /privacy-security/data-export |
| 10 | /dataerasure |

**POC of Configuring 2FA**



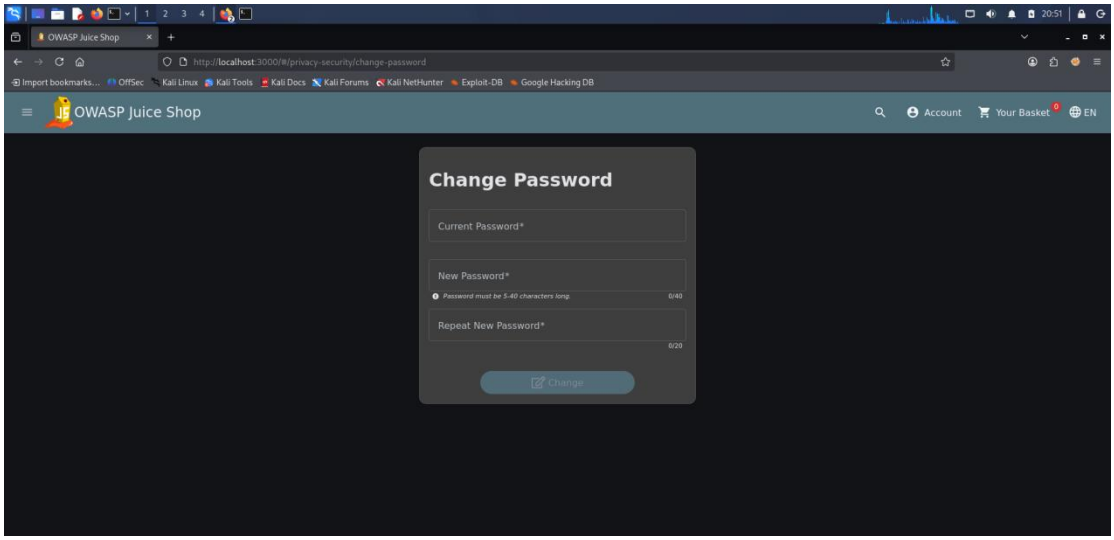

In the above figure it is shown that two-factor-authentication  page is accessed.

**POC of wallet**



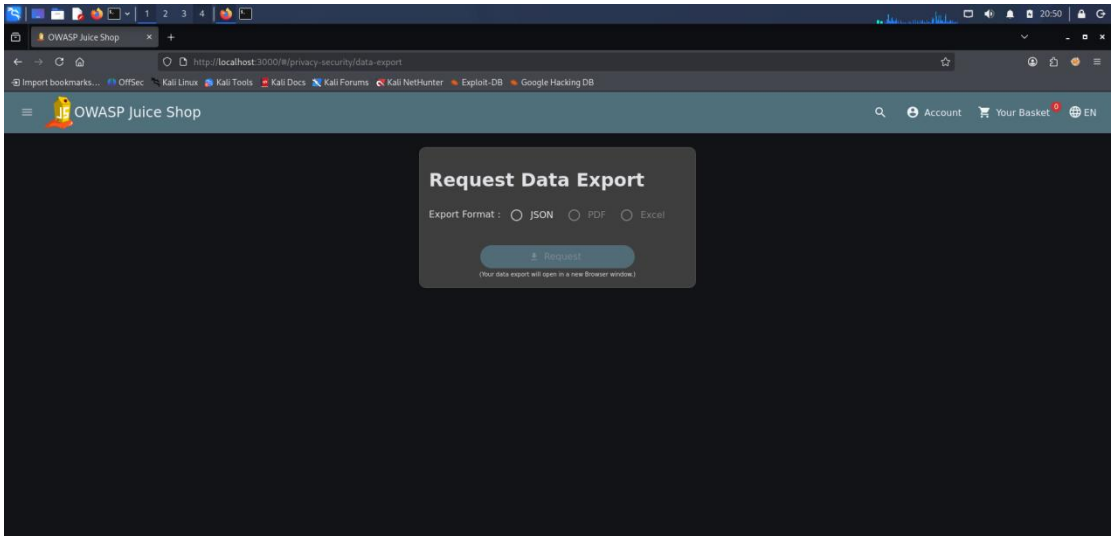

In the above figure it is shown that wallet page is accessed.

## POC of Change-Password



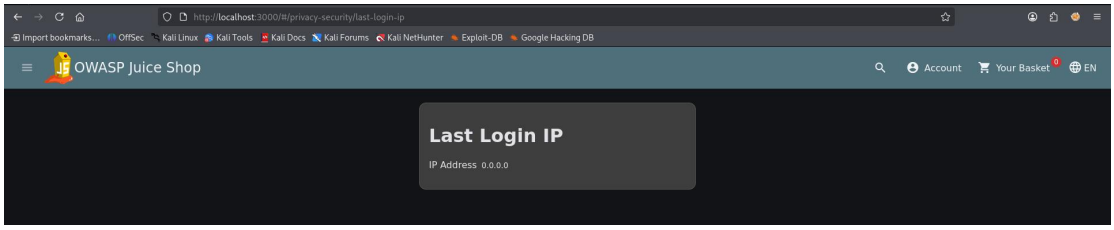

In the above figure it is shown that Change password page is accessed.

## POC of Data export




In the above figure it is shown that data export page is accessed.

## POC of Last login Ip



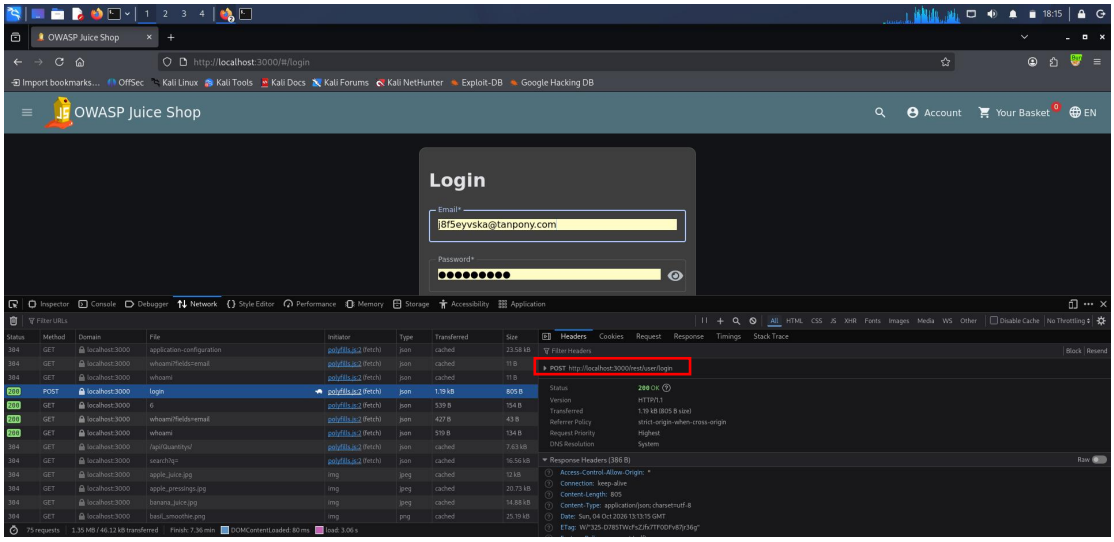

In the above figure it is shown that last login ip page is accessed.

# 2 API Discovery

| S.No | Method | Endpoint | Category |
|---|---|---|---|
| 1 | GET | http://localhost:3000/api/address | User/Address |
| 2 | GET | http://localhost:3000/rest/wallet/balance | User/wallet |
| 3 | GET | http://localhost:3000/api/cards | User/Payment |
| 4 | GET | http://localhost:3000/api/Feedback | Feedback |
| 5 | GET | http://localhost:3000/rest/captcha | Security/captcha |
| 6 | GET | http://localhost:3000/rest/user/login | Authentication |
| 7 | POST | http://localhost:3000/rest/order-history | User/Orders |
| 8 | GET | http://localhost:3000/api/Recycle | Application functionality |
| 9 | GET | http://localhost:3000/api/security/Gaurdian | Security/Authetnication |

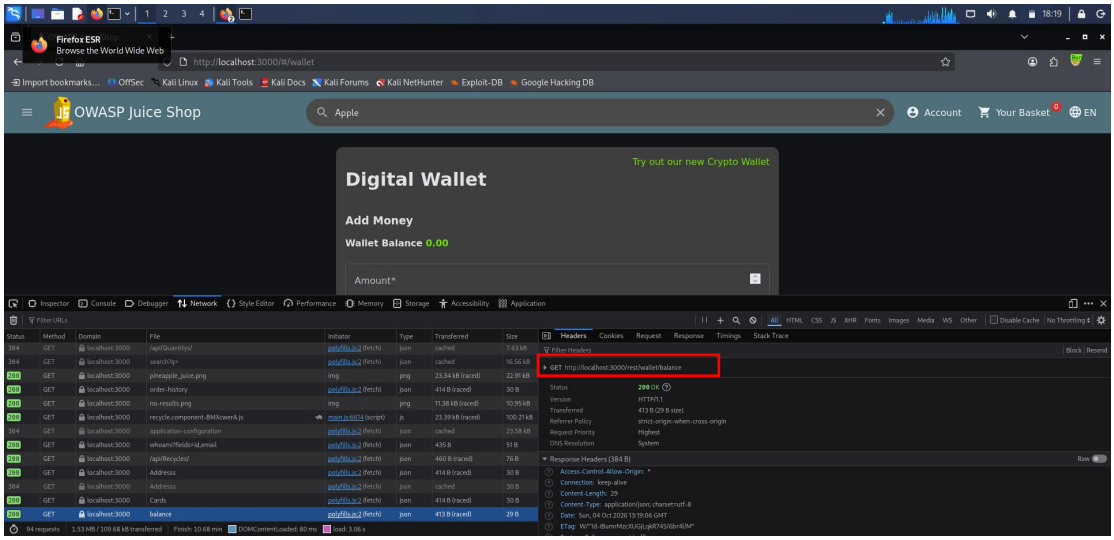
API Discovery involves identifying the application's exposed API endpoints, HTTP methods, and the functionality they provide. It is performed to understand how the web application communicates with its backend and to identify potential attack surfaces for further security testing. During this assessment, API endpoints were manually discovered by monitoring network requests through the browser's Developer Tools while interacting with different application features. The identified endpoints were documented along with their HTTP methods, functionality, and authentication requirements. The following screenshots provided below represent as an evidence of the API discovery process.

### POC of Login Page




In the above figure it is shown that api end point for login functionality is discovered using Browser dev tools.

### POC of Wallet Page

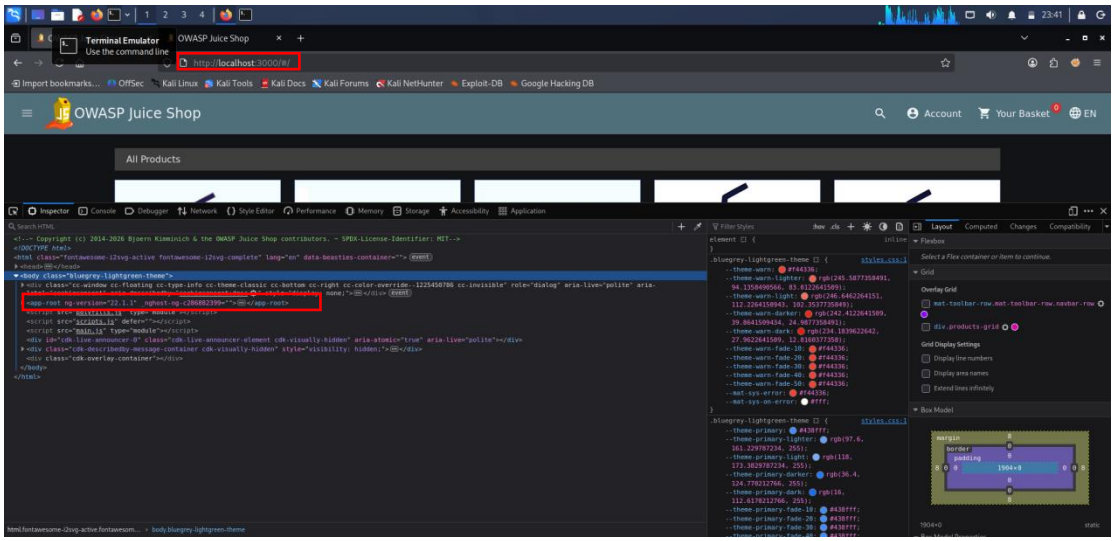



In the above figure it is shown that api end point for wallet functionality is discovered using Browser dev tools.

# 3 Technology Fingerprinting

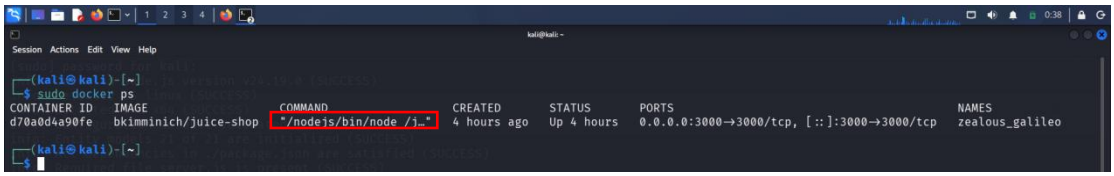
**Technology Fingerprinting** is the process of identifying the technologies, frameworks, libraries, servers, and databases used by a web application. For a penetration tester, this helps understand the application's technology stack and identify potential attack surfaces, known weaknesses, and areas that require further security testing.

## Front-end Framework




In the above Figure it is shown that The frontend framework was confirmed as Angular through direct inspection of the application's DOM using browser developer tools. The root element contains the Angular-specific **<app-root>** element and the ng-version="**22.1.1**" attribute. Additional Angular-specific indicators, including routerlink, **_nghost-***, and _ngcontent-* attributes, were also observed. The application uses Angular Router with **hash-based** client-side URLs such as #/.

## Backend-Technology




**In the above Figure it is shown that** The backend runtime was identified as Node.js based on the Docker container's startup command, which explicitly invokes /nodejs/bin/node. This provides direct evidence that the application is running on the Node.js runtime.

## Web server




The above figure shows that the web server is identified as **Express.js 4.22.1, running on the Node.js HTTP server**.

## Database technology



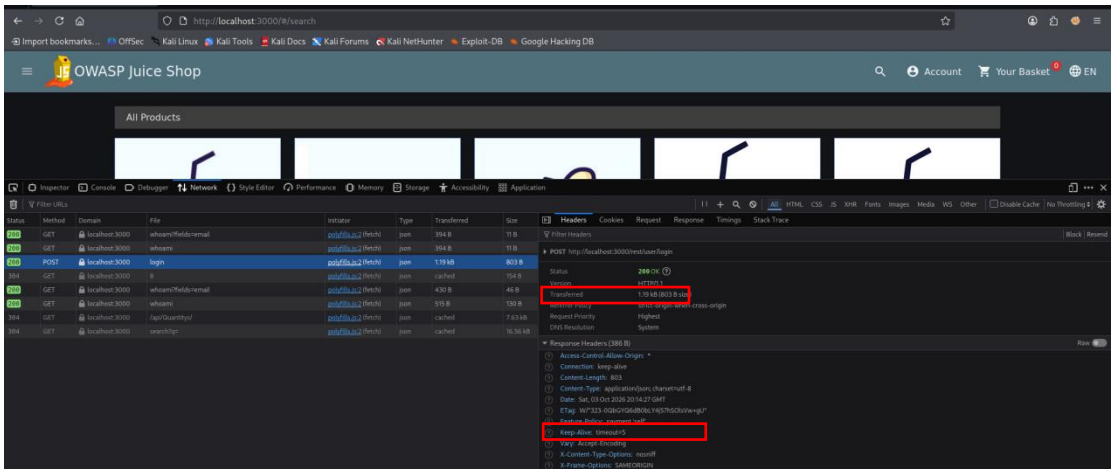

In the above Figure it is shown that The application's database technology was identified as **SQLite**. The application package configuration includes the sqlite3 package and Sequelize ORM, while inspection of the running Docker container confirmed the presence of the database file **juice-shop/data/juiceshop.sqlite**. This provides direct evidence that SQLite is used by the application.

## Javascript libraries



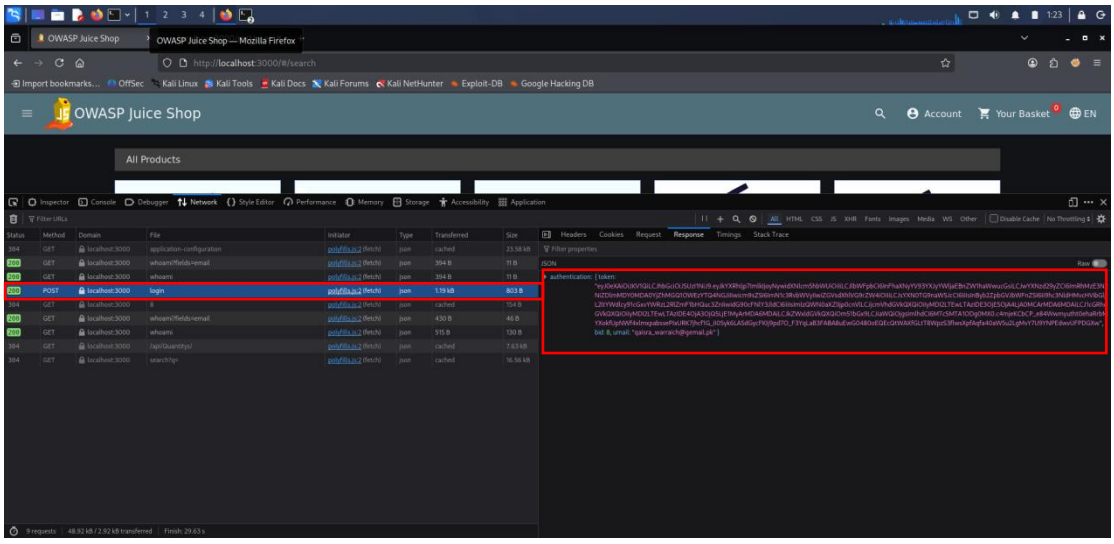

In the above Figure ti is shown that The Javascript libraries identified in application are Socket.IO 3.1.2 and Socket.IO Client 3.1.3 for real-time, bidirectional communication .

## API technologies




The above figure shows a **REST-style API endpoint** used for user authentication. The POST /rest/user/login endpoint accepts data using the application/json content type, demonstrating that the application communicates with its backend through a JSON-based REST API.

## Authentication mecahnism

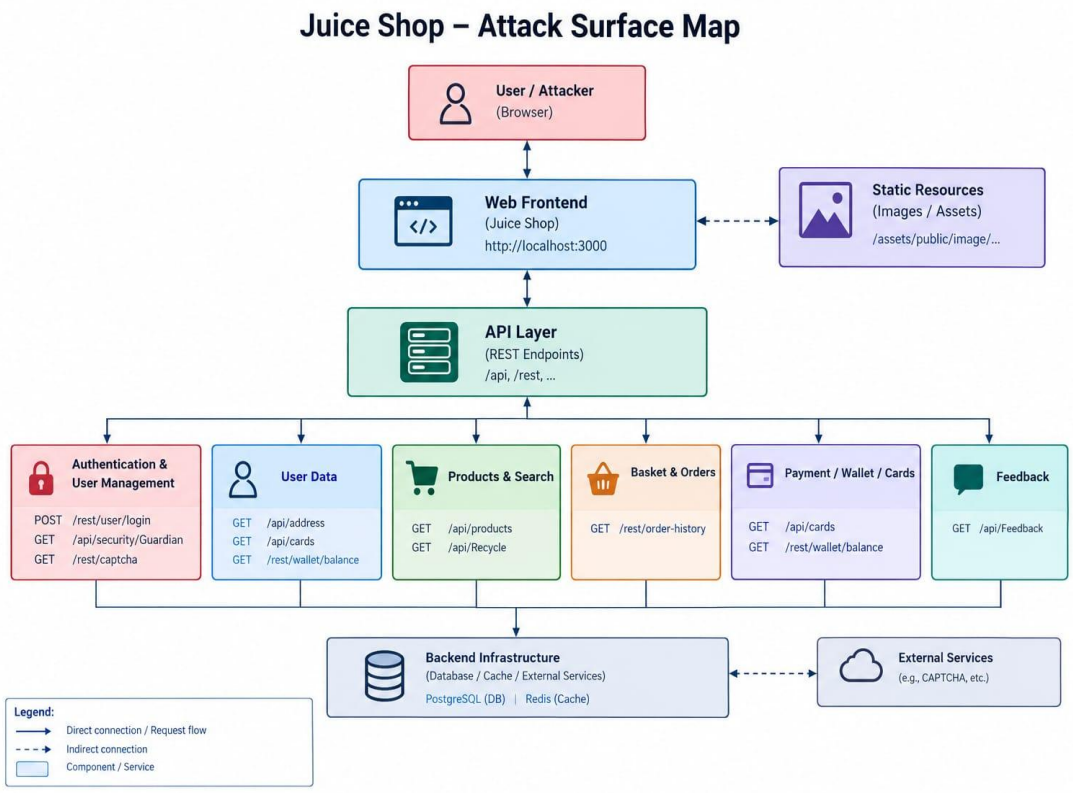



In the above Figure it is shown that The application uses **JSON Web Tokens (JWT) for token-based authentication**. The login response contains an authentication token whose decoded header identifies the token type as JWT and specifies RS256 as the signing algorithm. The token follows the standard three-part JWT structure consisting of a header, payload, and signature. For security and confidentiality, the complete authentication token has been redacted from the report.

**Technology Fingerprinting Table**

| S.No | Component | Identified Technology |
|---|---|---|
| 1 | Frontend Framework | Angular 22.1.1 |
| 2 | Ui/Icon Library | Material Icons |
| 3 | Backend Technology | Node.js |
| 4 | Web server | Express.js 4.22.1 |
| 5 | Database Technology | SQLite |
| 6 | ORM | Sequelize 6.37.3 |
| 7 | API Technology | REST API |
| 8 | Authentication mechanism | JWT |
| 9 | JavaScript Liraries | Socket.IO 3.1.2/ Socket.IO Client 3.1.3 |

# C) Build the Attack-Surface Map



Attack Surface Mapping was performed shown in above figure to consolidate the components and interfaces identified during application discovery into a single logical view. The map illustrates the relationship between the user, web frontend, API layer, and major application functions such as authentication, user data, products, orders, payment-related functionality, and feedback. The identified API endpoints and application resources were mapped to their respective functional areas to highlight potential interaction and testing points. This provides a structured overview of the application's exposed attack surface and serves as a basis for subsequent security testing.

# D) Attack Surface Inventory

| S.no | Asset/Endpoint | Type | Method | Access | Technology | Priority |
|---|---|---|---|---|---|---|
| 1 | /rest/user/login | Auth API | POST | Public | REST API | High |
| 2 | /rest/user/whoami | User API | GET | Authenticated | REST API | High |
| 3 | /api/address | User data API | GET | Authenticated | REST API | High |
| 4 | /api/cards | Payment data API | GET | Authenticated | REST API | Critical |
| 5 | /rest/wallet/balance | Wallet API | GET | Authenticated | REST API | High |
| 6 | /rest/order-history | Order API | GET | Authenticated | REST API | High |
| 7 | /api/Feedback | Feedback API | GET | Public | REST API | Medium |
| 8 | /rest/captcha | Security API | GET | Public | REST API | Medium |
| 9 | /api/security/Guardian | Security API | GET | Public | REST API | Medium |
| 10 | /api/Recycle | Application API | GET | Authenticated | REST API | Medium |
| 11 | /asset/public/image/products | Static resource | GET | Public | Web HTTP | Low |

# E) Prioritize the Attack Surface

**High Priority Areas**

- **/rest/user/login**: Investigate first because weaknesses in authentication, credential handling, brute-force protections, or account enumeration could affect every user account.

- **/rest/user/whoami**: Prioritize this endpoint because session and identity validation flaws could enable account confusion, session misuse, or unauthorized access.

- **/api/address**: Investigate because it exposes sensitive personal information and may reveal authorization weaknesses if one user can access another user’s address data.

- **/api/cards**: Treat as the highest-priority area because payment-related information is highly sensitive and unauthorized access could have serious privacy and financial consequences.

- **/rest/wallet/balance**: Investigate early because it handles financial information and may expose authorization or business-logic flaws affecting account balances.

- **/rest/order-history**: Prioritize because order histories may contain sensitive customer and transaction information and could be vulnerable to broken object-level authorization.

**Medium Priority Areas**

- **/api/Feedback**: Assess input validation, authorization behavior, and potential information disclosure because weaknesses could expose submitted feedback, user details, or internal application data.

- **/rest/captcha**: Evaluate whether CAPTCHA controls can be bypassed or abused to automate account-related activity, potentially weakening protections around authentication and other public endpoints.

- **/api/security/Guardian**: Review access controls and security-question handling because weaknesses could support account recovery or verification bypasses, potentially leading to unauthorized account access.

- **/api/Recycle**: Investigate input handling, access control, and application logic, particularly where the endpoint may affect stored or deleted data.

**Low Priority Areas**

**/assets/public/image/products/** Review for unintended file exposure, directory traversal, unsafe file handling, or disclosure of sensitive static resources. These assets appear less likely to directly affect authentication, authorization, or financial data.